AGENTIC AI - 1: BUILDING AGENTS WITH CODE 🤖

Name: _____ **Branch/Year:** _____ **Date:** _____

Warm-Up: Recap from Agentic AI - 1

1. What is the "Brain" of an AI Agent?

A. A database of facts

B. A Large Language Model (LLM) that predicts the next word

C. A search engine

D. A Python script

2. Why does an AI Agent need "Tools"?

A. LLMs can already do everything perfectly

B. LLMs can't do math, access live data, or perform actions on their own

C. Tools make the agent look cooler

D. Tools are required by law

3. What is the correct Agentic AI flow?

A. User → Tool → LLM → Answer

B. User → LLM (Think) → Tool (Execute) → LLM (Summarize) → Answer

C. User → Database → Answer

D. Tool → LLM → User → Answer

> **Today's Goal:** You will **build AI Agents with Python code** — step by step, from a bare LLM to an autonomous multi-agent system, learning the core concepts along the way.

Part 1: Setting Up the Brain in Code 🧠

To code an Agent, we need access to an LLM "brain". We'll use **Groq** — a free, blazing-fast API that runs models like LLaMA 3.3.

Analogy: Ordering Pizza vs. Cooking at Home

Using Groq API = Ordering pizza delivery. You don't need an oven (GPU). You call Groq, they "cook" the answer, and send it back. Fast and easy!

Using Ollama = Cooking at home. You need your own kitchen (powerful computer). More control, but more work.

Today, we order delivery (Groq) for speed. 🍕

Step 1: Install the Libraries

```
pip install langchain langchain-groq python-dotenv
```

Step 2: Connect to the Brain

```
from langchain_groq import ChatGroq

# Setup the "Brain" — LLaMA 3.3 running on Groq's servers
llm = ChatGroq(
    model="llama-3.3-70b-versatile",
    temperature=0,      # 0 = factual, 1 = creative
    api_key="YOUR_KEY"  # Get free key from console.groq.com
)
```

✏️ **ACTIVITY: Fill in the blanks:**

1. `temperature=0` makes the LLM more _____ (factual / creative).
2. `temperature=1` makes the LLM more _____ (factual / creative).
3. The model `"llama-3.3-70b-versatile"` has _____ billion parameters.
4. Getting a Groq API key is _____ (free / paid).

Part 2: The Problem — LLMs Can't Do Math! 🤯

Let's prove that the "Brain" alone has limits. Ask it a hard math question:

```
# Ask the Brain a hard math question
question = "What is 34567 multiplied by 89123?"

response = llm.invoke(question)
print(f"LLM's Answer: {response.content}")
```

🔍 **OBSERVATION:** The LLM will give a number that *looks* right but is almost certainly **wrong**. It's a word-predictor, not a calculator! The actual answer is **3,080,714,741**.

✏️ **ACTIVITY: Why does this happen?** Choose the best explanation:

A. The LLM is broken and needs to be updated

B. The LLM predicts tokens (words/numbers) statistically — it doesn't actually calculate

C. The numbers are too big for the computer's memory

D. The API has a calculator bug

💡 **KEY INSIGHT:** This is exactly why we need **Tools**. The Brain is smart at language, but dumb at math. Solution? Give it a calculator!

Part 3: Creating a Tool (Giving the Brain Hands) 🛠️

Analogy: The Smart Student with a Calculator

Imagine a brilliant literature student who can write essays about anything — but can't do long division. You hand them a calculator. Now they can write *and* compute!

LLM = The brilliant student | **Tool** = The calculator

Step 1: Define the Tool (Just a Python Function!)

```
from langchain_core.tools import tool

@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers together, accepting floats."""
    return a * b
```

✏️ **ACTIVITY: Read the code above and answer:**

1. What does the `@tool` decorator do?

_____

Hint: *It tells LangChain: "This function can be used by the AI agent as a tool."*

2. Why does the function need a **docstring** (`"""Multiply two numbers..."""`)?

_____

Hint: *The LLM reads the docstring to understand WHEN to use this tool.*

Step 2: Bind the Tool to the Brain

```
# Give the LLM access to our calculator tool
llm_with_tools = llm.bind_tools([multiply])
```

`bind_tools()` is like handing the calculator to the student and saying: "*You can use this whenever you need to do math.*"

Step 3: Ask the Same Question Again

```
response = llm_with_tools.invoke(question)

# The LLM doesn't answer directly — it asks to use the tool!
print(f"Content: {response.content}")
print(f"Tool Calls: {response.tool_calls}")
```

🔍 **OBSERVATION:** Notice something amazing: `response.content` is **empty**! The LLM didn't guess this time. Instead, `response.tool_calls` contains: `[{'name': 'multiply', 'args': {'a': 34567, 'b': 89123}}]`. The brain *decided on its own* to use the calculator!

✏️ **ACTIVITY: Trace the Flow:** Number these steps 1-4 in the correct order:

_____ The `multiply` tool runs and returns the exact answer
_____ User asks: "What is 34567 * 89123?"
_____ LLM reads the question and decides it needs the `multiply` tool
_____ LLM receives the tool result and presents the final answer to the user

Part 4: The Manual Runtime (You Play the Manager) 🧑‍💼

Right now, the LLM only *requests* a tool call — it doesn't actually run the function. **Someone** needs to execute it. That someone is called the **Runtime**.

Analogy: The Intern and the Boss

The LLM (intern) says: "*Boss, I need to use the calculator for 34567 × 89123.*"

You (the runtime/boss) pick up the calculator, press the buttons, and give the answer back to the intern.

In the code below, **YOU** are manually being the boss:

```
# YOU are the Runtime here!
tool_calls = response.tool_calls

if tool_calls:
    action = tool_calls[0]            # Get the request
    func_name = action['name']        # "multiply"
    args = action['args']             # {'a': 34567, 'b': 89123}

    if func_name == "multiply":
        result = multiply.func(args['a'], args['b'])
        print(f"Calculated Answer: {result}")
else:
    print("AI answered directly:", response.content)
```

🔍 **OBSERVATION:** Notice we wrote `multiply.func(args['a'], args['b'])` — not just `multiply(...)`.
Why?
Because the `@tool` decorator **wraps** our original function inside a LangChain `Tool` object. The variable `multiply` is no longer a plain Python function — it's a Tool wrapper.
To call the **original Python function** directly, we access it via `.func`. Think of it like this:

- `multiply` = the wrapped Tool object (used by the agent framework)
- `multiply.func` = the actual Python function underneath (used when YOU call it manually)

✏️ **ACTIVITY: Fill in the blanks for understanding:**

1. `action['name']` returns the _____ of the tool the LLM wants to use.
2. `action['args']` returns the _____ the LLM wants to pass to the tool.
3. The problem with this manual approach is: _____

Hint: *What if the LLM needs to call multiple tools, or use one tool's output as input for another?*

💡 **KEY INSIGHT:** Writing manual if/else for every tool doesn't scale. What if you have 10 tools? What if the agent needs to call tools **in a loop**? We need an **automated runtime**. Enter: **LangGraph**.

Part 5: LangGraph — The Automated Agent Loop 🔄

Analogy: The Autopilot Assembly Line 🏭

Imagine a factory with two stations: a **Thinker** (decides what to build) and a **Builder** (actually builds it). A robot arm keeps moving parts between them until the product is done.

LangGraph = The robot arm that automates the loop.

The 3 Key Concepts

Concept	What it is	Analogy
State	A shared notebook that holds the conversation history	A whiteboard everyone can read & write on
Nodes	Workers that do something (LLM node, Tool node)	Stations on the assembly line
Edges	Rules for who goes next	The conveyor belt between stations

Step 1: Define the State (Shared Notebook)

```
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph.message import add_messages

class State(TypedDict):
    messages: Annotated[list, add_messages]
# Translation: "State has a 'messages' list.
# When new info comes, ADD it, don't overwrite."
```

Step 2: Define the Nodes (Workers)

```
from langgraph.prebuilt import ToolNode

# Worker 1: The Assistant (asks LLM to think)
def assistant_node(state: State):
    response = llm_with_tools.invoke(state["messages"])
    return {"messages": [response]}

# Worker 2: The Tool Runner (executes tool calls)
tool_node = ToolNode([multiply])
```

Step 3: Define the Router (Traffic Cop)

```
from langgraph.graph import END

def should_continue(state: State):
    last_msg = state["messages"][-1]
    if last_msg.tool_calls:    # LLM wants a tool?
        return "tools"         # → Go to tool node
    return END                 # → Done, stop the loop
```

Step 4: Build & Compile the Graph

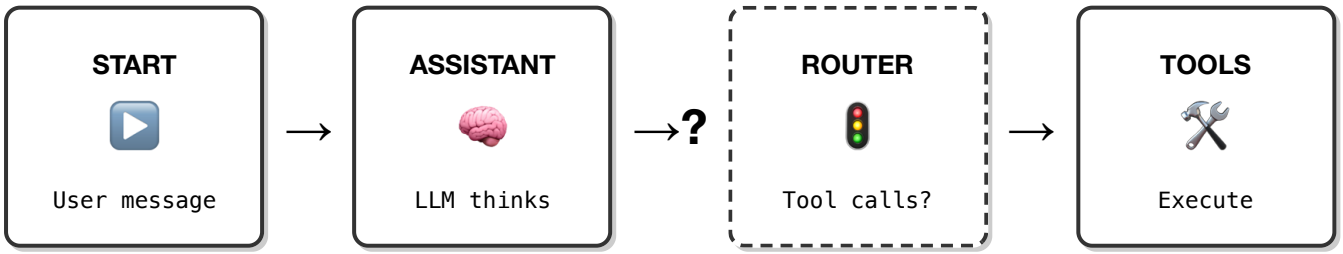
```
from langgraph.graph import StateGraph, START

builder = StateGraph(State)
builder.add_node("assistant", assistant_node)
builder.add_node("tools", tool_node)

builder.add_edge(START, "assistant")        # Start → Assistant
builder.add_conditional_edges("assistant", should_continue)
builder.add_edge("tools", "assistant")      # Tools → back to Assistant

graph = builder.compile()
```

✏️ **ACTIVITY: Graph Flow Activity:** Draw arrows showing the flow. Fill in what happens at each step:



If router says "tools" → go to Tools → loop back to Assistant.
If router says "END" → stop and return final answer.

Step 5: Run the Agent!

```
query = "Calculate 50 times 173. Then divide that result by 5."
result = graph.invoke({"messages": [("user", query)]})
print(result["messages"][-1].content)
```

🔍 **OBSERVATION:** The agent will: (1) call `multiply(50, 173)` → get 8650, then (2) need to divide by 5 — but wait, we don't have a `divide` tool! The smart LLM figures out that dividing by 5 is the same as multiplying by 0.2, so it calls `multiply(8650, 0.2)` → get 1730.0, then (3) presents the final answer. **This is why we used `float` in our tool — the LLM needs decimals to work around missing tools!**

💡 **KEY INSIGHT: Pro Tip:** Building graphs manually is great for learning. In practice, for simple agents (one LLM + tools), LangGraph provides a one-liner shortcut: `create_react_agent(llm, tools=[multiply])` — it auto-creates the State, Nodes, Edges, and Router we just built! Import it from `langgraph.prebuilt`.

Part 6: Building a Real-World Agent — Crypto Price Checker 💰

Let's build something useful: an agent that checks **live cryptocurrency prices** from the internet!

Step 1: Create a Web Tool

```
import requests
from langchain_core.tools import tool

@tool
def get_crypto_price(coin_id: str) -> float:
    """Fetches current price of a cryptocurrency in USD.
    Args: coin_id: e.g., 'bitcoin', 'ethereum'"""
    url = f"https://api.coingecko.com/api/v3/simple/price?ids={coin_id}&vs_currencies=usd"
    data = requests.get(url).json()
    return data[coin_id]['usd']
```

Step 2: The Persona Concept (The "Soul")

We can give any LLM a **personality** using a `SystemMessage`. This is like giving your intern a job description before they start:

```
from langchain_core.messages import SystemMessage

system_instruction = """
You are a ruthless, high-energy Wall Street trader.
Speak in financial slang ('bullish', 'bearish', 'to the moon').
When you give a price, analyze if it's high or low.
"""
sys_msg = SystemMessage(content=system_instruction)

# Test it with a simple question (no tools)
response = llm.invoke([sys_msg, ("user", "How is the tech market?")])
print(response.content)  # Responds in trader personality! 🤑
```

✏️ **ACTIVITY: Design Your Own Persona:** Write a system instruction for a different agent personality:

Agent Name: _____
System Instruction:

YOUR CUSTOM PERSONA

Hint: *Example ideas: A pirate captain financial advisor, a zen monk weather reporter, a sarcastic movie critic code reviewer.*

Step 3: Run the Crypto Agent

```
tools = [get_crypto_price, multiply]
llm_with_tools = llm.bind_tools(tools)

# Build graph (same pattern as Part 5)
# ... (same builder code) ...

initial_state = {
    "messages": [
        ("user", "I own 2.5 bitcoin. Check the live price and tell me my portfolio value in
USD.")
    ]
}
result = graph.invoke(initial_state)
print(result["messages"][-1].content)
```

🔍 **OBSERVATION:** The agent will: (1) call `get_crypto_price("bitcoin")` to get the live price, (2) call `multiply(price, 2.5)` to calculate portfolio value, then (3) present the final answer. **Two different tools, fully automated!**

💡 **KEY INSIGHT: Why does the query say "in USD"?** Because our tool hardcodes `vs_currencies=usd`. If you ask for INR or EUR, the agent will fail — the tool simply doesn't support other currencies! **An agent is only as good as its tools.** To support multiple currencies, you would need to add a `currency` parameter to the tool function.

⚠️ **WARNING:** **Persona + Tools Don't Always Mix!**
You might try adding `sys_msg` into `initial_state["messages"]` to make the agent respond in the trader persona. However, on some providers (like Groq), combining a `SystemMessage` with tool-calling can cause a `BadRequestError` — the persona confuses the LLM's tool-call format.
Takeaway: Personas work great with plain LLM calls (Step 2 above). When using agents with tools, test persona compatibility with your provider first!

Part 7: Multi-Agent Systems — The AI Software House 🏢

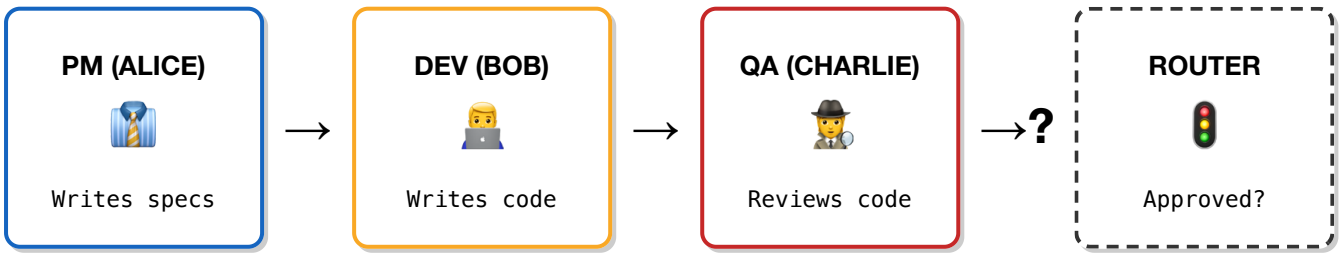
Analogy: A Real Software Company

In a real company, one person doesn't do everything. You have:

- **Product Manager (PM)** — converts user requests into specs
- **Developer (Dev)** — writes the code
- **QA Engineer** — reviews and rejects bad code

What if **each role** was a separate AI agent, and they **collaborated automatically**?

The Architecture



If QA rejects → loop back to Dev | If QA approves → END

Section 1 of 4: The Shared State (The Whiteboard)

In a real office, a team uses a shared whiteboard to track progress. Each person reads it and updates it. In our system, the **State** is that whiteboard — a Python dictionary that all agents can read from and write to.

`TypedDict` is a Python feature that lets us define *exactly* what fields our dictionary has, like a form with fixed fields:

```
from typing import TypedDict

class SoftwareState(TypedDict):
    user_request: str      # What the user wants to build
    requirements: str      # PM fills this with the spec
    code: str             # Developer fills this with code
    qa_feedback: str      # QA fills this with review notes
    iteration_count: int  # Tracks how many Dev-QA cycles
    status: str           # "APPROVED" or "REJECTED"
```

> ✏️ **ACTIVITY: Think about it:** Why do we need `iteration_count`?
>
> Hint: *What would happen if QA always rejects? The loop would run forever, costing infinite API calls!*

Section 2 of 4: The Agent Functions (The Workers)

Each agent is a simple Python function that: (1) reads the State, (2) does its job using the LLM, (3) returns updated fields. Let's understand each one:

Agent 1: The Product Manager (PM)

The PM reads the user's request and converts it into a technical specification (a detailed plan for the developer).

```
def pm_node(state):
    print("[PM] Analyzing request...")
    prompt = f'Create a technical spec for: "{state["user_request"]}"'
    response = llm.invoke(prompt)
    print(f"[PM] Spec ready! ({len(response.content)} chars)")
    return {"requirements": response.content}
```

Line-by-line:

- `state["user_request"]` — reads what the user wants from the whiteboard
- `llm.invoke(prompt)` — asks the LLM to create a spec from that request
- `return {"requirements": ...}` — writes the spec back to the whiteboard under "requirements"

Agent 2: The Developer (Dev)

The Dev reads the PM's spec (and QA feedback if code was rejected before) and writes Python code.

```
def dev_node(state):
    iteration = state["iteration_count"] + 1
    if state.get("qa_feedback"):
        print(f"[DEV] Rewriting code (attempt #{iteration})...")
        context = f"PREVIOUS CODE REJECTED. FEEDBACK: {state['qa_feedback']}. Fix it."
    else:
        print(f"[DEV] Writing initial code (attempt #{iteration})...")
        context = ""
    prompt = f"Write Python code for: {state['requirements']}. {context}"
    response = llm.invoke(prompt)
    print(f"[DEV] Code written! ({len(response.content)} chars)")
    return {"code": response.content, "iteration_count": iteration}
```

Line-by-line:

- `state.get("qa_feedback")` — checks if QA rejected before. `.get()` returns `None` safely if the field doesn't exist yet (first time there's no feedback)
- If feedback exists → Dev sees the complaints and rewrites. If not → Dev writes from scratch

- `return {"code": ..., "iteration_count": iteration}` — writes code + incremented counter to whiteboard

Agent 3: The QA Engineer

QA reads the Dev's code, reviews it, and stamps it APPROVED or REJECTED.

```
def qa_node(state):
    print("[QA] Reviewing code...")
    prompt = f"Review this code: {state['code']}. Say APPROVED or REJECTED with reasons."
    response = llm.invoke(prompt)
    status = "APPROVED" if "APPROVED" in response.content else "REJECTED"
    print(f"[QA] Verdict = {status}")
    return {"qa_feedback": response.content, "status": status}
```

Line-by-line:

- `state['code']` — reads the Dev's code from the whiteboard
- The prompt asks the LLM to review and say APPROVED or REJECTED
- `"APPROVED" if "APPROVED" in response.content` — checks if the word "APPROVED" appears in the review text. This is a simple string-matching trick!
- `return {"qa_feedback": ..., "status": ...}` — writes the review and verdict to the whiteboard

> ✏️ **ACTIVITY: Pattern Recognition:** All 3 agents follow the same pattern. Fill in the blanks:
>
> 1. Read from _____ (the shared whiteboard)
> 2. Build a _____ (question for the LLM)
> 3. Call _____ (ask the LLM)
> 4. Return a _____ with updated fields
>
> Hint: *Answers: state / prompt / llm.invoke(prompt) / dictionary*

Section 3 of 4: The Router (The Traffic Cop)

After QA reviews the code, *someone* needs to decide: **ship it or send it back?** That's the Router — a function that looks at the current state and returns where the graph should go next.

```
def router(state):
    if state["status"] == "APPROVED":
        return END              # QA approved! Stop the loop.

    if state["iteration_count"] > 3:
        return END              # Safety: max 3 attempts

    return "developer"          # Rejected! Send back to Dev.
```

How it works:

- Returns `END` → the graph **stops** (pipeline finished)
- Returns `"developer"` → the graph **goes back** to the dev_node (this creates the loop!)
- The safety check (`> 3`) prevents infinite loops — imagine a QA that rejects everything forever!

Section 4 of 4: Building the Graph (Wiring the Team)

Now we connect everything together. Think of this as drawing arrows between workers on a flowchart:

```
from langgraph.graph import StateGraph, START, END

workflow = StateGraph(SoftwareState)

# Register the 3 agents as nodes (workers)
workflow.add_node("pm", pm_node)
workflow.add_node("developer", dev_node)
workflow.add_node("qa", qa_node)

# Draw the arrows (edges) between them
workflow.add_edge(START, "pm")            # Entry point: PM goes first
workflow.add_edge("pm", "developer")      # PM's output goes to Dev
workflow.add_edge("developer", "qa")      # Dev's output goes to QA

# QA's output goes to the Router (conditional)
workflow.add_conditional_edges(
    "qa", router,
    {"developer": "developer", END: END}
)

app = workflow.compile()
print("Software House ready!")
```

Key functions explained:

Function	What it does	Example
`StateGraph(SoftwareState)`	Creates a blank graph that uses our State format	Like creating an empty flowchart
`add_node("name", func)`	Registers a worker function with a name	`add_node("pm", pm_node)`
`add_edge("from", "to")`	Draws a fixed arrow: always go from A to B	`add_edge("pm", "developer")`
`add_conditional_edges("from", func, map)`	Like a traffic signal at a junction: after "from" finishes, it calls `func(state)` which returns a string (e.g., "developer" or END). The `map` then routes that string to the matching node.	`add_conditional_edges("qa", router, {"developer": "developer", END: END})` — after QA, call `router()`. If it returns "developer" → go to Dev. If END → stop.
`compile()`	Locks in the graph and makes it ready to run	Like pressing "Save" on your flowchart

⚠️ **WARNING:** **Colab Users: Run all 4 sections in ONE cell!**
In Google Colab, each code block typically becomes a separate cell. But Section 4 references names from Sections 1-3 (`SoftwareState` , `pm_node` , `dev_node` , `qa_node` , `router` , `END`). If you run them as separate cells, you risk `NameError` when cells are run out of order or when the kernel restarts. **Copy all 4 code blocks into a single Colab cell** to ensure everything is defined together and runs without errors.

✏️ **ACTIVITY: Understand the System:** Answer these questions:

1. What happens when QA sets `status = "APPROVED"` ? → _____
2. What does `add_edge("pm", "developer")` mean? → _____

3. What is the difference between `add_edge` and `add_conditional_edges` ?

Hint: `add_edge` = *always go there.* `add_conditional_edges` = *a function decides where to go.*

Step: Run the Simulation!

```
# Give the team a project!
print("STARTING THE SOFTWARE HOUSE")
print()

initial_state = {
    "user_request": "Create a snake game in Python with a scoring system.",
    "iteration_count": 0,
    "status": "IN_PROGRESS"
}

# stream() shows each step as it happens!
for step in app.stream(initial_state):
    node_name = list(step.keys())[0]
    print(f"  >> Node '{node_name}' finished.")

print()
print("SIMULATION COMPLETE")

# Get the final output
final = app.invoke(initial_state)
print()
print("FINAL CODE:")
print(final["code"])
```

🔍 **OBSERVATION:** Using `app.stream()` instead of `app.invoke()` lets you **watch each node finish in real-time**. You'll see output like:

```
[PM] Analyzing request...
  >> Node 'pm' finished.
[DEV] Writing initial code (attempt #1)...
  >> Node 'developer' finished.
[QA] Reviewing code...
[QA] Verdict = REJECTED
  >> Node 'qa' finished.
[DEV] Rewriting code (attempt #2)...
  >> Node 'developer' finished.
[QA] Verdict = APPROVED
  >> Node 'qa' finished.
```

Now you can **see** the loop: PM → Dev → QA (reject) → Dev again → QA (approve) → END!

💡 **KEY INSIGHT:** This is a **Multi-Agent System**! Three AI "employees" collaborate in a loop. The PM writes specs, the Dev codes, the QA reviews — and if the code is bad, it loops back to the Dev automatically. All without any human intervention!

Part 8: Terminology Consolidation 📚

Match the Term to Its Definition

Write the letter of the correct definition next to each term:

#	Term	Ans	#	Definition
1	`@tool`	_____	A	Connects a tool function to the LLM so it knows it can use it
2	`bind_tools()`	_____	B	A shared data store passed between nodes in a graph
3	`State`	_____	C	Python decorator that converts a function into an agent-usable tool
4	`ToolNode`	_____	D	Instructions defining the agent's personality and behavior
5	`SystemMessage`	_____	E	A graph framework where you define State, Nodes, and Edges to build an agent
6	`StateGraph`	_____	F	Pre-built graph node that automatically executes tool function calls

Part 9: Final Challenge — Design Your Own Agent 🚀

Mission: Blueprint a Custom Agent

You've learned all the building blocks. Now, design an agent on paper. Pick **one** scenario from below (or invent your own):

Option	Agent Idea	Tools Needed
A	Fitness Coach Agent — calculates BMI, suggests exercises	BMI calculator, exercise database lookup
B	News Summarizer Agent — fetches today's headlines and summarizes them	News API, text summarizer
C	Code Review Agent — reviews code and suggests improvements	Linter tool, documentation checker

Your choice: _____ (or custom idea: _____)

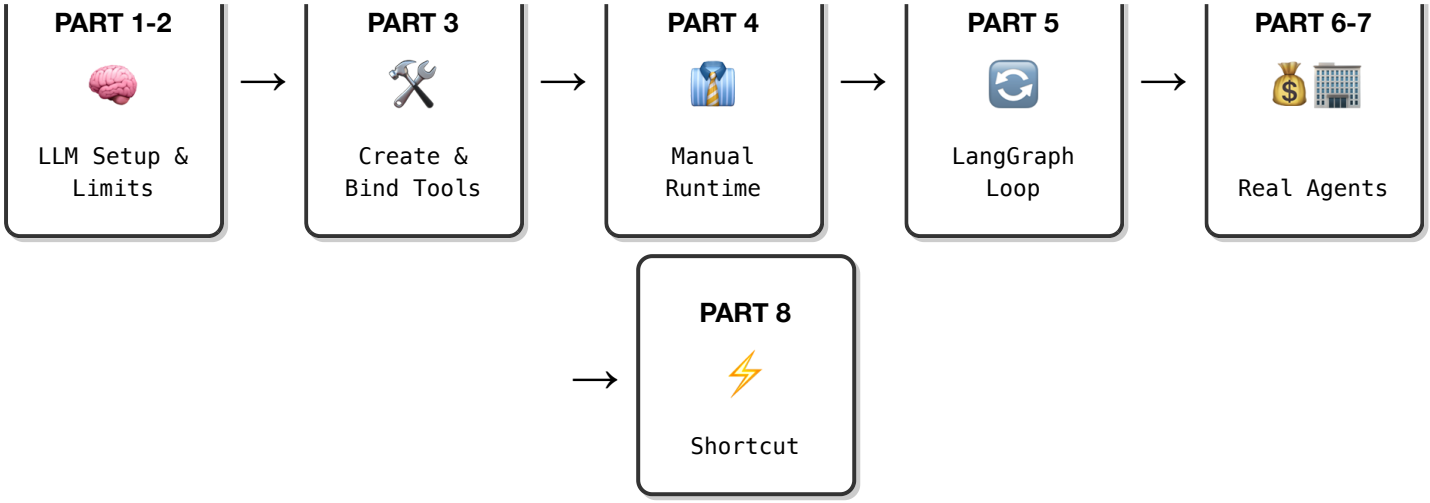
1. WHAT TOOLS DOES YOUR AGENT NEED? (LIST 2-3 AS PYTHON FUNCTIONS)

2. WHAT PERSONA/SYSTEM MESSAGE WOULD YOU GIVE IT?

3. DRAW THE GRAPH FLOW (NODES AND EDGES)

4. WHAT SHOULD THE ROUTER'S LOGIC BE? (WHEN TO LOOP, WHEN TO STOP?)

Recap: Your Learning Map 🗺️



💡 **KEY INSIGHT: The Big Picture:** An AI Agent = LLM (Brain) + Tools (Hands) + Graph (Automation). You started with a bare LLM that couldn't do math, gave it tools, automated the loop with LangGraph, and ended up with multi-agent systems that can simulate entire teams!